Models Rmd

Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify
```

```
## { T }
```

Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, immitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y is between 0 and 1, which represents 16,000 years ago to 2,400 years ago.
myFunc <- function(x) {
case_when(
    0 <= x & x <= (-9+ 16)/(13.6)                          ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x  & x <= (-7.500 + 16)/(13.6)   ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x   & x <= (-4.3+ 16)/(13.6)       ~ 3,
    (-4.3 + 16)/(13.6) < x   & x <= (-3.000+ 16)/(13.6)    ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x   & x <= (-2.400+ 16)/(13.6)    ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE                                                    ~ 1
  )
}
curve(myFunc(x), col = 2, main = "Relative Sea Level and Year")

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
    T[i]<-rpois(1,  Y[i]* 25)
    if(T[i] < 1) {T[i]<- 1}
```

```
    else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
    SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL, main = "Synthetic Data Plot(T and SL)")
plot(Y, T, main = "Synthetic Data Plot(Y and T)")
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5
```

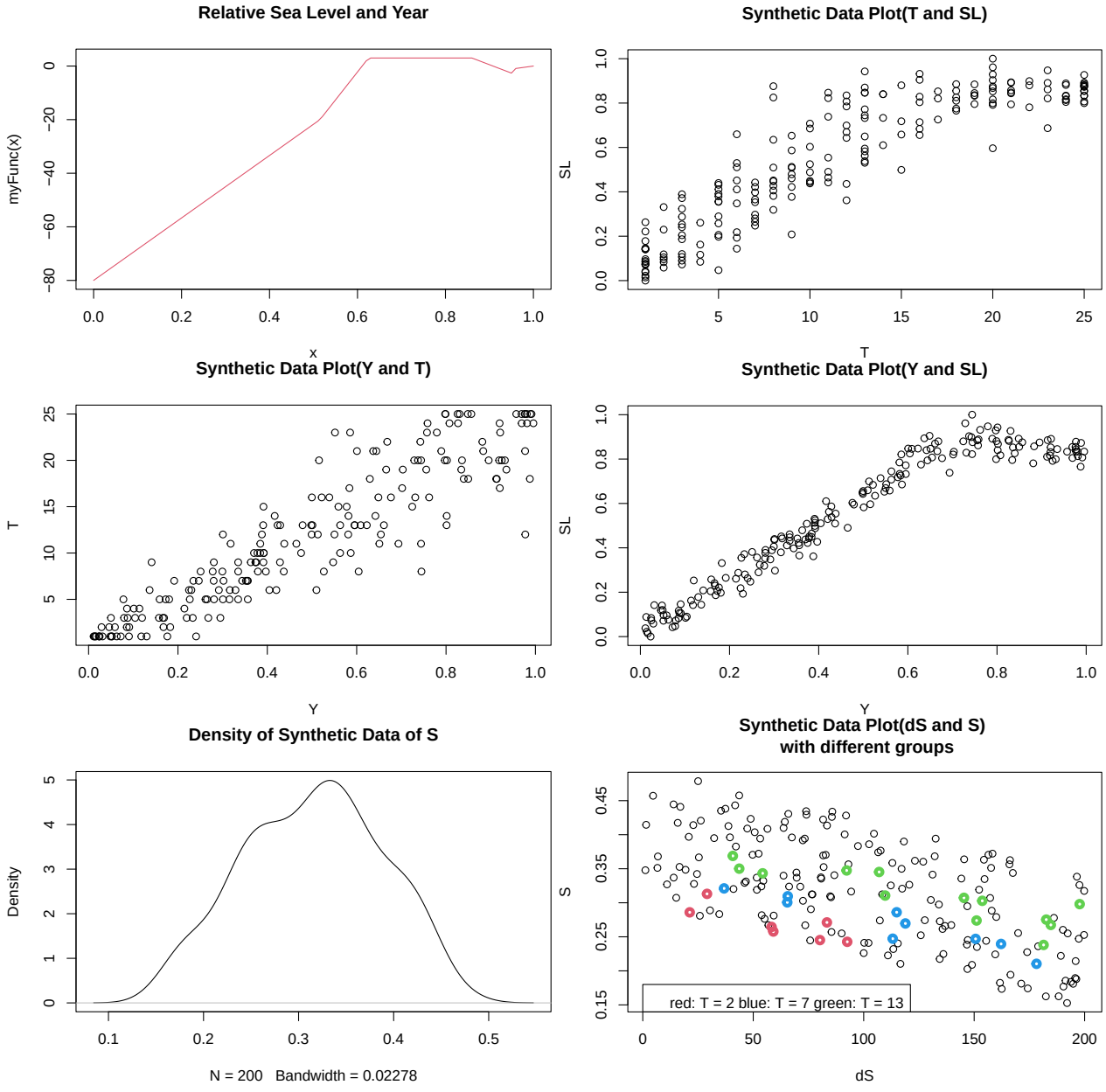
```
## [1] 0.1527208 0.4786818
```

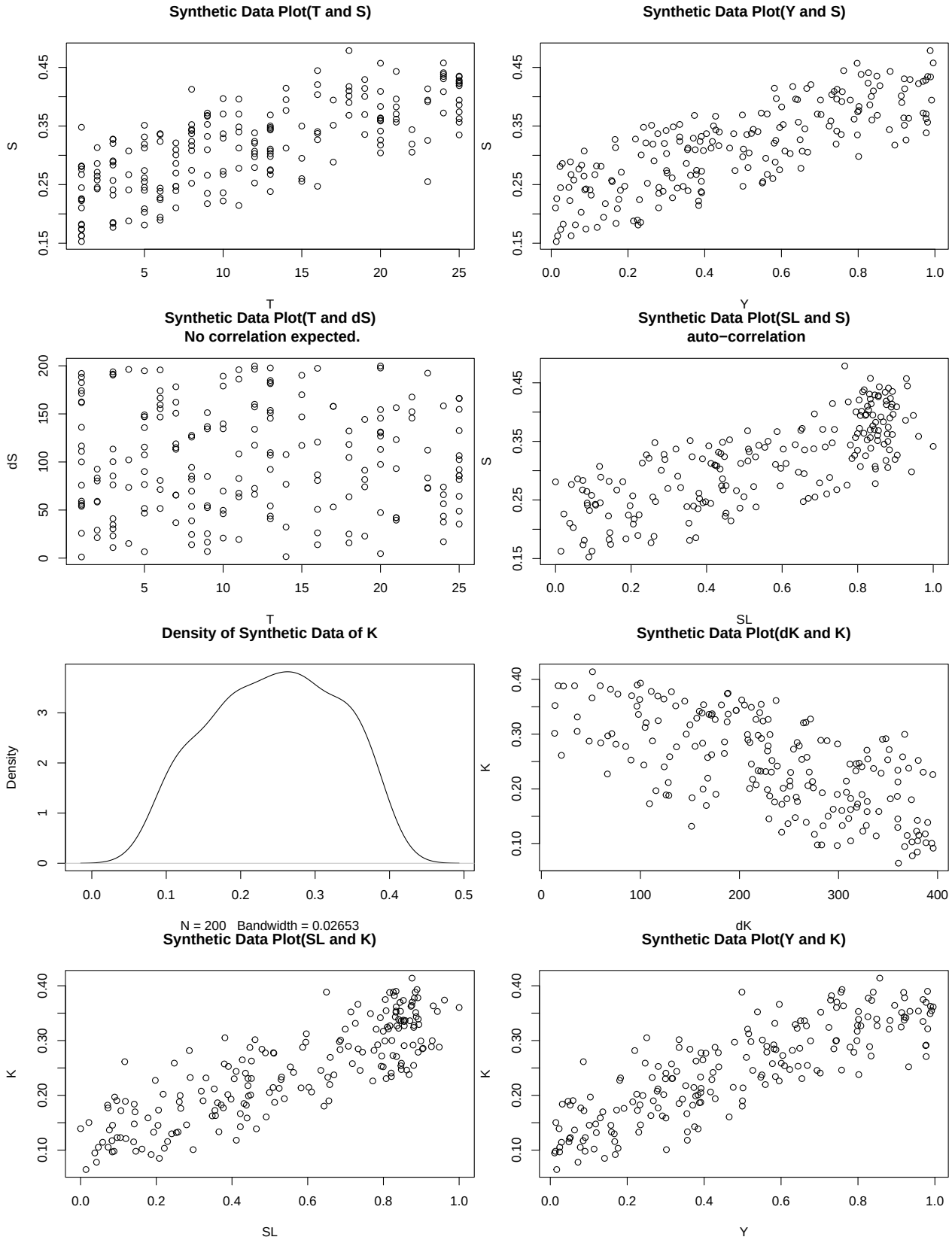
```
plot(density(S), main = "Density of Synthetic Data of S")

plot(dS, S, main = "Synthetic Data Plot(dS and S) \n with different groups")
legend(x = 0, y = 0.18, legend="red: T = 2 blue: T = 7 green: T = 13")
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S, main = "Synthetic Data Plot(T and S)")
plot(Y, S, main = "Synthetic Data Plot(Y and S)")
plot(T, dS, main = "Synthetic Data Plot(T and dS)\n No correlation expected.") # no correlation
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i]- dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K)# try to make less than 0.5
```

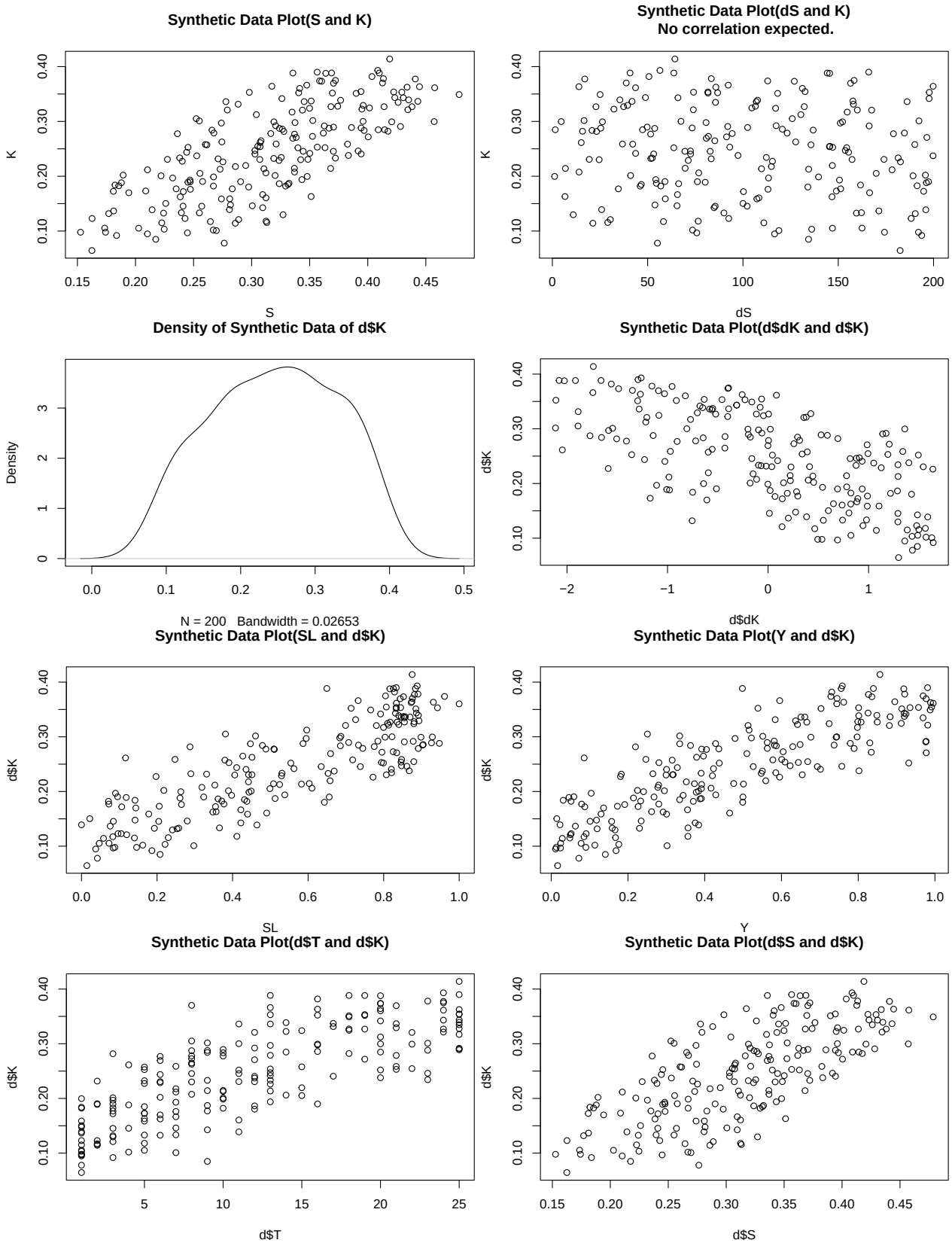
```
## [1] 0.06449945 0.41389870
```

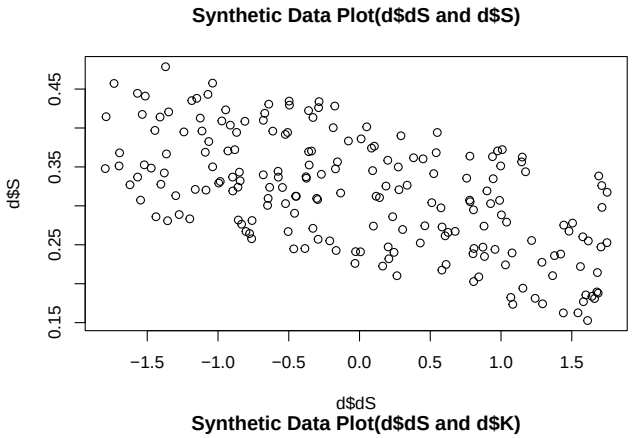
```
plot(density(K), main = "Density of Synthetic Data of K")
plot(dK, K, main = "Synthetic Data Plot(dK and K)")
plot(SL, K, main = "Synthetic Data Plot(SL and K)")
plot(Y, K, main = "Synthetic Data Plot(Y and K)")
plot(S, K, main = "Synthetic Data Plot(S and K)")
plot(dS, K, main = "Synthetic Data Plot(dS and K)\n No correlation expected.") # no correlation

# Standardize data without SL, S, K (those are ratio)
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))
```

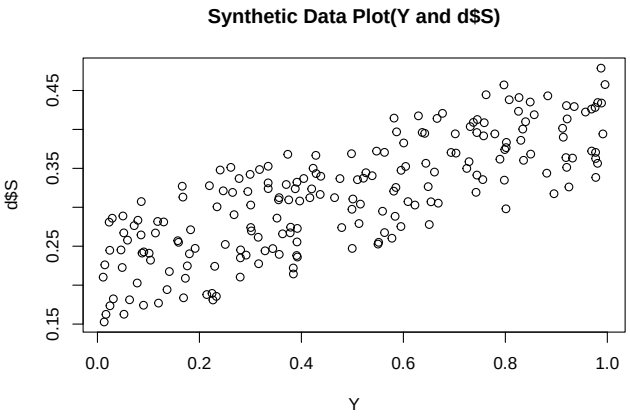
```r
# check the data
plot(density(d$K), main = "Density of Synthetic Data of d$K")
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
plot(Y, d$K, main = "Synthetic Data Plot(Y and d$K)")
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
plot(Y, d$S, main = "Synthetic Data Plot(Y and d$S)")
plot(d$dS, d$K, main = "Synthetic Data Plot(d$dS and d$K)\n No correlation expected.") # no correlation
```

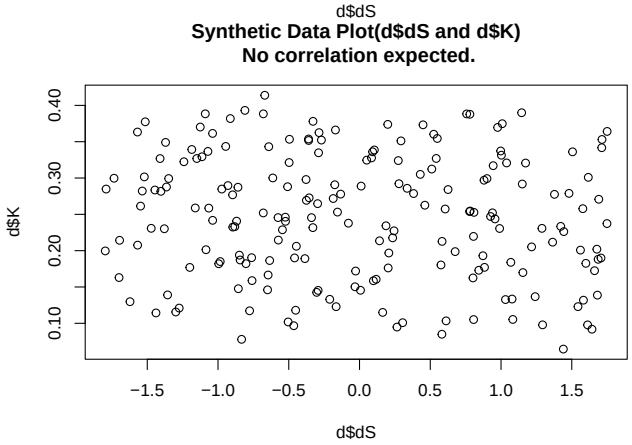



Synthetic Data Plot(T and S)
S
T
Synthetic Data Plot(Y and S)
S
Y
Synthetic Data Plot(T and dS)
No correlation expected.
dS
T
Synthetic Data Plot(SL and S)
auto−correlation
S
SL
Density of Synthetic Data of K
Density
N = 200 Bandwidth = 0.02653
Synthetic Data Plot(dK and K)
K
dK
Synthetic Data Plot(SL and K)
K
SL
Synthetic Data Plot(Y and K)
K
Y


Synthetic Data Plot(S and K)
K
S
Synthetic Data Plot(dS and K)
No correlation expected.
K
dS
Density of Synthetic Data of d$K
Density
N = 200 Bandwidth = 0.02653
Synthetic Data Plot(d$dK and d$K)
d$K
d$dK
Synthetic Data Plot(SL and d$K)
d$K
SL
Synthetic Data Plot(Y and d$K)
d$K
Y
Synthetic Data Plot(d$T and d$K)
d$K
d$T
Synthetic Data Plot(d$S and d$K)
d$K
d$S



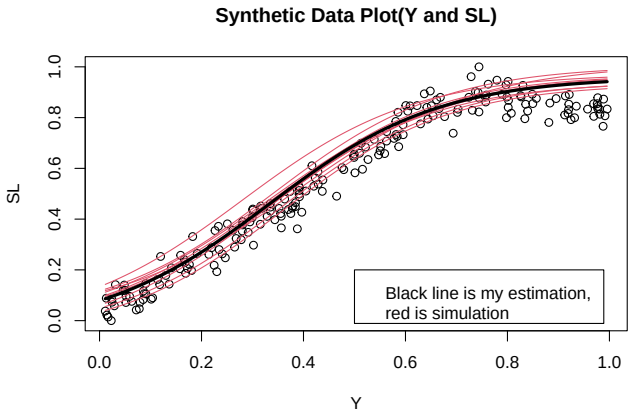




Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and Sl are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 6, 0.2)*x + rnorm(1, -2, 0.2)) - rnorm(1, 0.04, 0.02), add =TRUE
curve(inv_logit(6*x - 2) - 0.04, add =TRUE, lwd = 3)
legend(0.5, 0.2,legend="Black line is my estimation, \nred is simulation")
```



Model 1

- Also see overview.pdf to know statistical model and prior
- This model stratifies all parameters by T.
- There are a few messages to warning my priors. Since it only shows up during first iteration of warmup, we can safely ignore it.
- I just omit it for m1.2 and 1.3.

```
# Model 1
m1<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

        # We do not know anything about this but think similar for all group.
        # So use partial pooling to regularize the estimate.
        # mean should start at 0 and has small variation.

        # In this case (m1), we do not include correlation.
        # This is because multilevel models of S and SL includes the information of Y
        # But we have to regularize.
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        z_bK_Y[T] ~ normal(0,0.1),

        # don't need regularization as dK is not affected by T
        # We are hoping/believing b_dK is negative, but this is not prior.
        bK_dK ~ normal(0, 0.3),
         U ~ exponential(1),

     # Model S
         S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS,

         # I am hoping/believing b_dS is negative and b_Y_S is positive
         # partial pooling for the same reason for a2 and b_Y_S
         vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
         vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
         c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
         c(sigma_aS, sigmabS_Y) ~ exponential(6),
         z_aS[T] ~ normal(0,0.1),
         zbS_Y[T] ~ normal(0,0.1),

        # don't need regularization
         bS_dS ~ normal(0,0.3),
         sigma_S ~ exponential(1),
```

```
    # Model SL by Y
        # The log function and prior was refered to the previous research of Sea Level Change over time
        # This function is abstruct so I will have to actually stan code to provide accurate estimates
        # ulam code in r does not allow us to hartd code the conditional function.
        # vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal(0.5, 0.3)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1", # SL is between 0 and 1
        Y    = "lower=0,upper=1" # Y is between 0 and 1
        #, T    = "lower=1,upper=25"  #this is unnecessary since T is provided in the dat


    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1843:3: note:   by 'void ulam_cm
##  1843 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1843:3: note:   by 'void ulam_cm
##  1843 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crt
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1880:3: note:   by 'T_ ulam_cmdstanr_e
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::lo
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1880:3: note:   by 'T_ ulam_cmds
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1880:3: note:   by 'T_ ulam_cmds
```

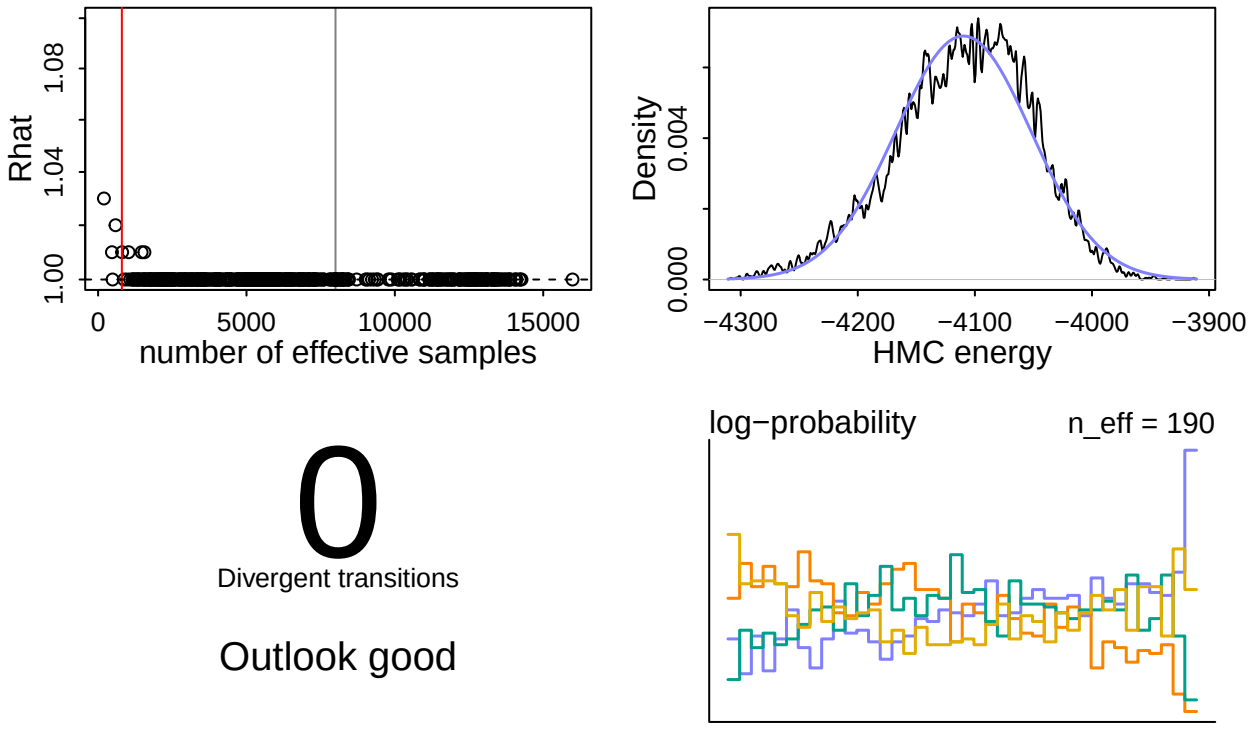
```
## const'
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c3e5d2199.hpp:1880:3: note:   by 'T_ ulam_cmds
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
```

```
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
```

```
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
```

```
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 55.5 seconds.
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 60.3 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 60.4 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 61.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 59.3 seconds.
## Total execution time: 61.1 seconds.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m1)
```



- This has so much divergent and less effective sampling.
- This would be due to the large number of parameters, including hidden estimated ones (SL and Y). I will reduce parameters to reliably interpret the result.
  - First one would be Y as S and SL are more important for the model.
    - by the way, SL and Y in model K change the efficiency in the highest rate.

```
# Model 1.2
m1.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
        c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
```

```
        c(sigma_aS, sigmabS_Y) ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        zbS_Y[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),

        # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

        # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

        # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1"
        , Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

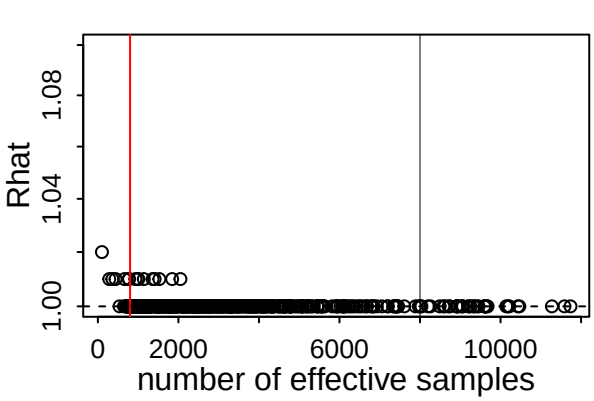
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
```

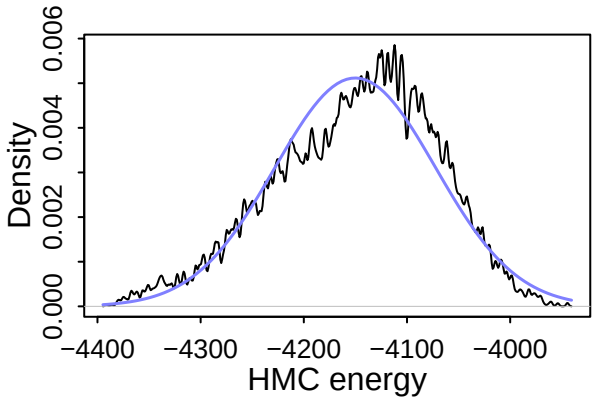
```
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
```

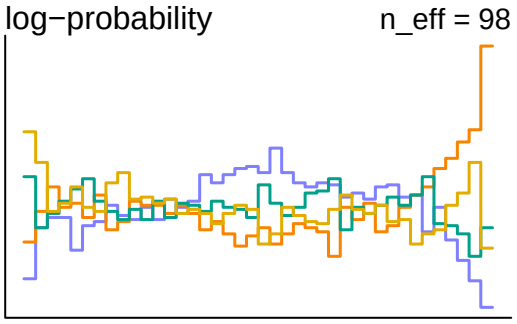
```
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 36.7 seconds.
```

```
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 40.2 seconds.
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 50.2 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 50.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 44.4 seconds.
## Total execution time: 50.7 seconds.
```

```
dashboard(m1.2)
```







This is still bad model as the eff is very small number

```
# Model 1.3
m1.3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i]  + bK_dK * dK,

        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_dS * dS,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        aS_bar ~ normal(0,0.3),
        sigma_aS ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),
```

```
    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y     = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

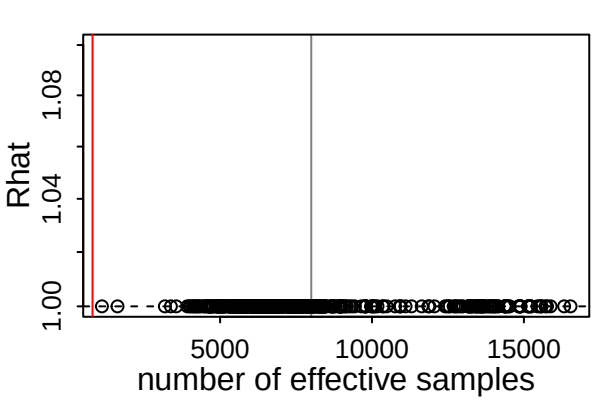
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

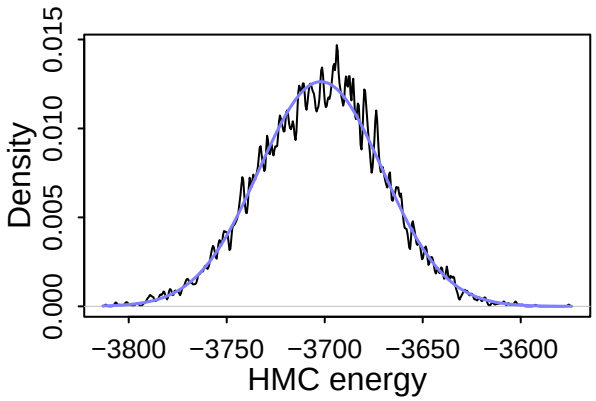
```
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
```

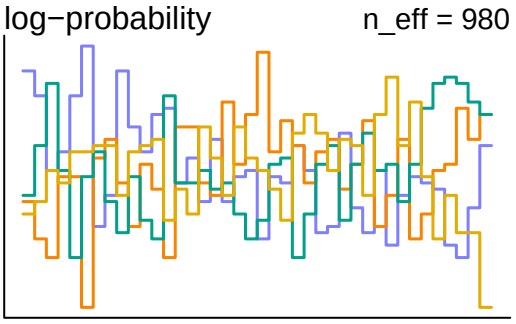
```
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
```

```
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 27.8 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 29.3 seconds.
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 33.2 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 34.6 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 31.2 seconds.
## Total execution time: 34.7 seconds.
```

```
dashboard(m1.3)
```








```
precis(m1.3, depth=1)
```

```
##                   mean          sd         5.5%       94.5%     rhat  ess_bulk
## bK_SL_bar    1.11350434 0.069859495  1.001387211  1.22503793 1.001464  1622.132
## bK_S_bar     1.10128548 0.190810847  0.787842216  1.40533057 1.000602  3919.145
## aK_bar      -2.17797776 0.052911275 -2.261745472 -2.09064768 1.000786  3556.261
## sigma_bK_SL  0.14883866 0.131722191  0.008562868  0.40280433 1.000773  3378.506
## sigma_bK_S   0.16319605 0.157254979  0.010081270  0.46134189 1.000375  6835.174
## sigma_aK     0.13250741 0.113100250  0.008864479  0.34645975 1.000846  3182.104
## bK_dK       -0.22361414 0.012095345 -0.242931071 -0.20434995 1.000183  3989.647
## U            0.01949632 0.002333940  0.015815182  0.02324584 1.001973  1109.727
## aS_bar      -0.76059796 0.034510144 -0.814878394 -0.70466040 1.000774  4493.010
## sigma_aS     0.99478194 0.240884700  0.644002435  1.40187428 1.001149  4654.520
## bS_dS       -0.20171910 0.018562373 -0.231604974 -0.17213315 1.001038 12938.332
## sigma_S      0.05226078 0.002783976  0.048029426  0.05686444 1.000404  8971.526
```

- This is very high eff and no divergent. This is great. This model is reliably sampled to be used for analysis!

Posterior Check

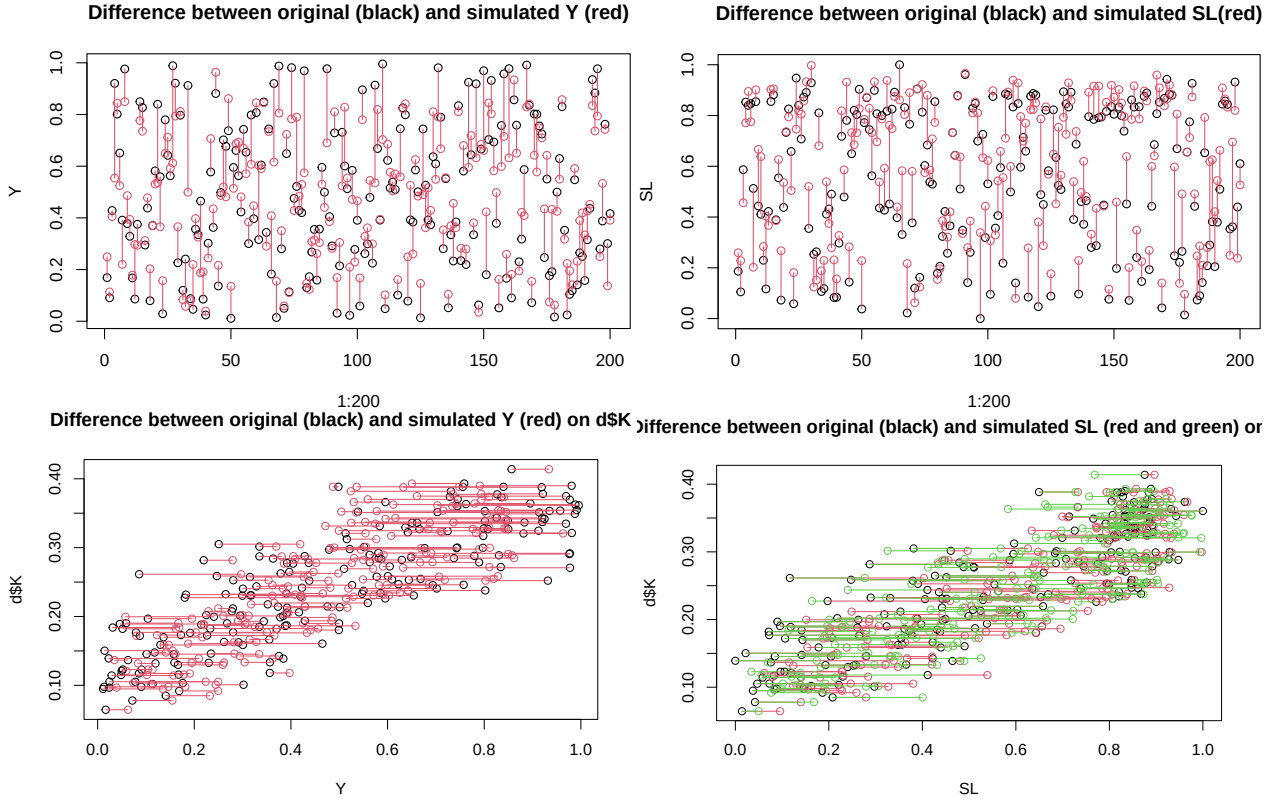
```
post <- extract.samples(m1.3)
```

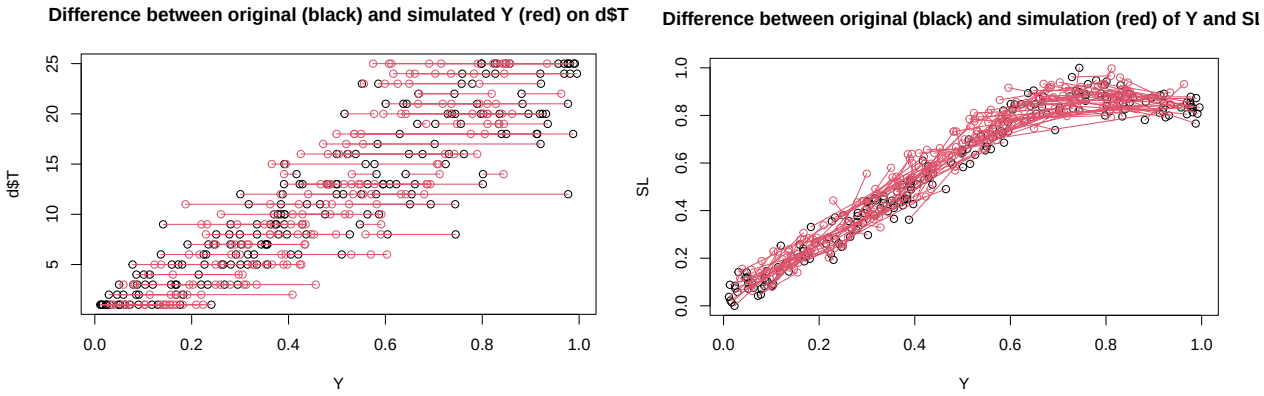
Hidden parameters

- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y, main = "Difference between original (black) and simulated Y (red)")
points(1:200, post$Y[1,], col = 2)
```

```r
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

plot(1:200, SL, main = "Difference between original (black) and simulated SL(red)")
points(1:200, post$SL[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$SL[1,i], SL[i]), col = 2)

# check Y and SL with K
plot(Y, d$K, main = "Difference between original (black) and simulated Y (red) on d$K")
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

plot(SL, d$K, main = "Difference between original (black) and simulated SL (red and green) on d$K")
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

# Y and T
plot(Y, d$T, main = "Difference between original (black) and simulated Y (red) on d$T")
points(post$Y[1,], d$T ,col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i],d$T[i]), col = 2)

# Y and SL both hidden parameters
plot(Y, SL, main = "Difference between original (black) and simulation (red) of Y and SL")
points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)
```

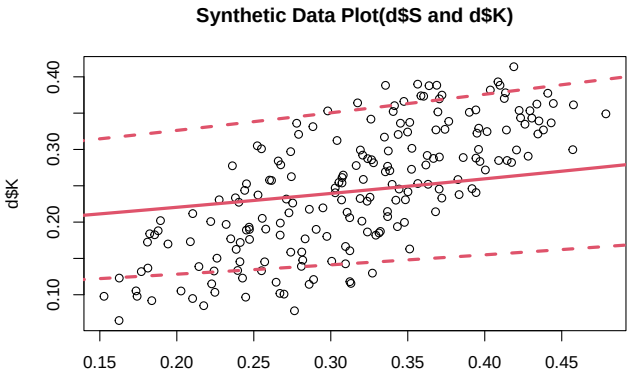


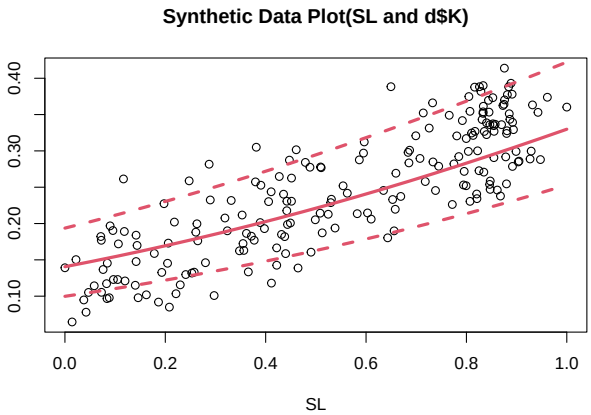


=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthetized) data.

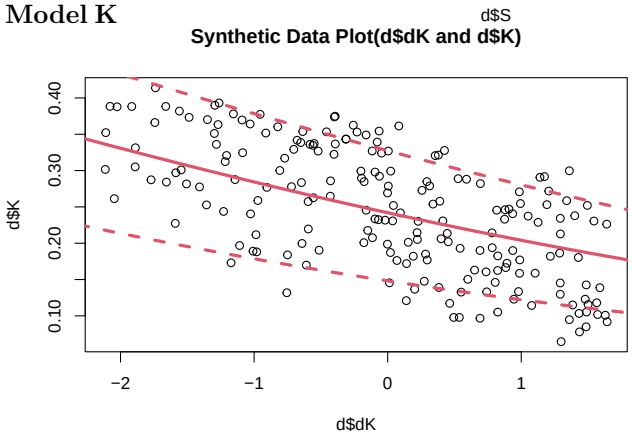
Posterior Simulation

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
    function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$dK
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```





Model K



- Looks nice!

Model S

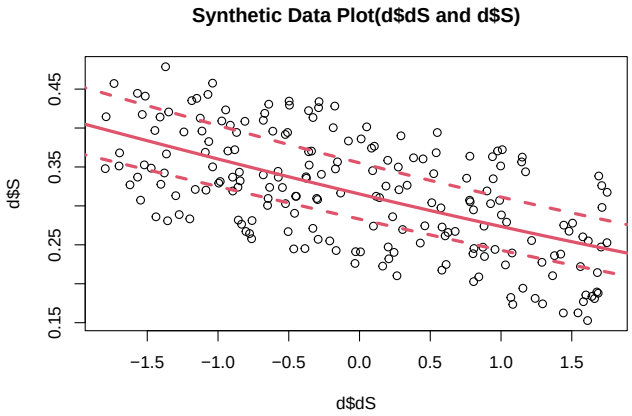
- Do the same thing for S
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided in the data.

```
post$SL[1,] == post$S[1,]
```

```
##   [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

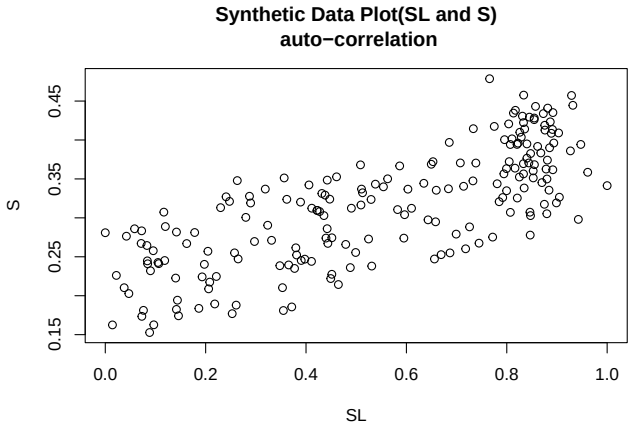
```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
```

```
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



- Functions catch the trend very well!
- The model 1.3 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.
- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

```
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")
```



This is because there is a correlation!!

Model 2 (m2)

- see priors in overview.pdf, but this include correlation between SL and S
- This is based on the Ch 14.3 in Statistical Rethinking as SL and S are not centred
- warnings are annoying but relatively few so I just omit it for m2.2 and m.3.

```
# Model 2
m2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,
```

```
    # We do not know anything about each parameter but think similar for all group.
    # So use partial pooling to regularize estimates.
    # mean should start at 0 and has small variation.

    # In this case (m2), we do include correlation between S and SL due to Y.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_Y_bar)~normal(0,0.3),
    c(sigma_aK, sigma_bK_Y) ~ exponential(6),
    z_aK[T]~normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    transpars>vector[25]:bK_S <<- v[,1],
    transpars>vector[25]:bK_SL <<- v[,2],

    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1), # I do not expect very very strong corre
    matrix[2,25]: zSSL ~ normal(0 , 1),

    # don't need regularization
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

# Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,

    # I am hoping/believing b_dS is negative and b_Y_S is positive
    # partial pooling for the same reason for a2 and b_Y_S
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    zbS_Y[T] ~ normal(0,0.1),

    # don't need regularization
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),

# Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
    T ~ poisson(Y * 25),

# Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
    SL    = "lower=0,upper=1", # SL is between 0 and 1
    Y    = "lower=0,upper=1" # Y is between 0 and 1
```

```
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1789:3: note:   by 'void ulam_cm
##  1789 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1789:3: note:   by 'void ulam_cm
##  1789 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::lo
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1828:3: note:
## by 'T_ ulam_cmdstanr_faadbf2fa06d1ae2933d92a379944cca_model_namespace::ulam_cmdstanr_faadbf2fa06d1ae
##  1828 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1828:3: note:   by 'T_ ulam_cmds
##  1828 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1828:3: note:   by 'T_ ulam_cmds
## trix<T_a, -1, 1>&, std::ostream*) const'
##  1828 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c2fc62e83.hpp:1828:3: note:   by 'T_ ulam_cmds
##  1828 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
```

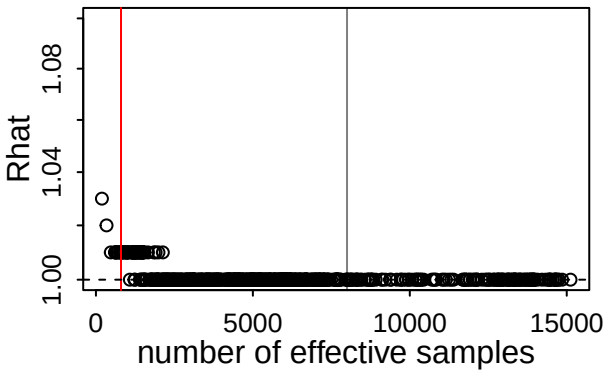
```
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 1
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 3
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Use
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
```

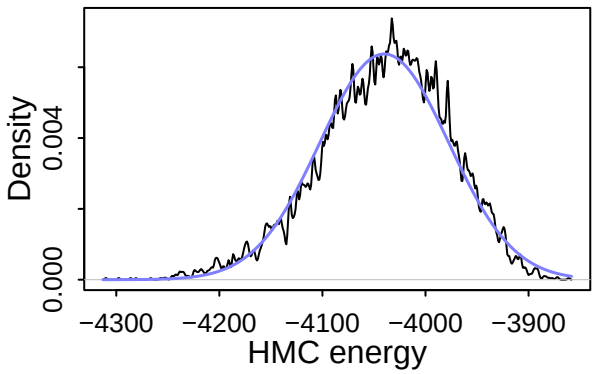
```
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
```

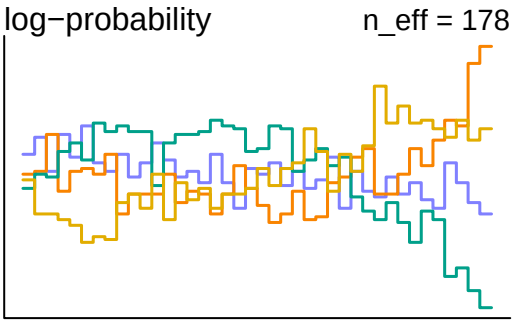
```
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 89.9 seconds.
```

```
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 100.4 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 101.8 seconds.
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 122.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 103.7 seconds.
## Total execution time: 123.0 seconds.

## Warning: 37 of 8000 (0.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m2)
```







-> Looks bad sampling efficiency.

Therefore, I will tackle this problem with the same approach though I do not want to remove the correlation parameters.

```
# Model 2,2
m2.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        aK_bar~normal(0,0.3),
        sigma_aK ~ exponential(6),
        z_aK[T]~normal(0,0.1),

        transpars>vector[25]:bK_S <<- v[,1],
        transpars>vector[25]:bK_SL <<- v[,2],

        transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
        vector[2]: sigmaSSL ~ exponential(2),
        cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
        matrix[2,25]: zSSL ~ normal(0 , 1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
        c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
```

```
        c(sigma_aS, sigmabS_Y) ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        zbS_Y[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

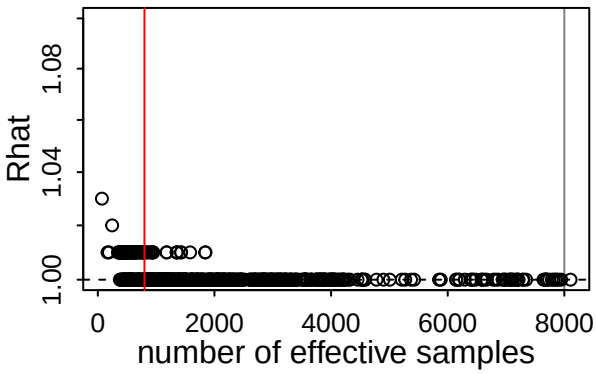
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

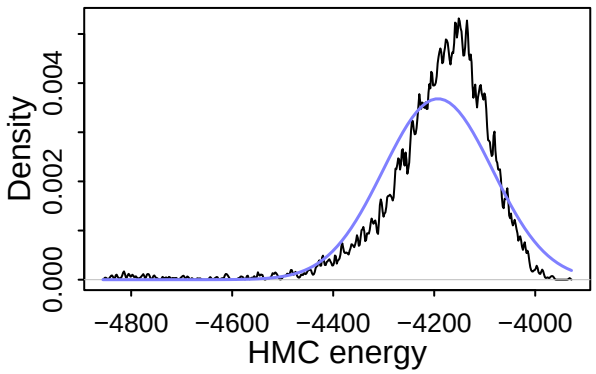
```
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
```

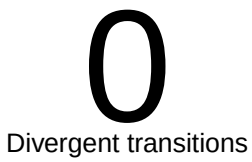
```
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
```

```
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 211.3 seconds.
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 225.0 seconds.
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 236.5 seconds.
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 1364.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 509.3 seconds.
## Total execution time: 1364.4 seconds.
```

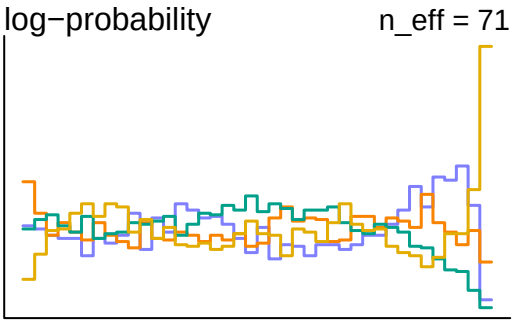
```
dashboard(m2.2)
```







Outlook good



- This sampling efficiency is still not great. Let me remove one more thing so that it corresponds m1.3.

```
# Model 2.3
m2.3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        aK_bar~normal(0,0.3),
        sigma_aK ~ exponential(6),
        z_aK[T]~normal(0,0.1),
        transpars>vector[25]:bK_S <<- v[,1],
        transpars>vector[25]:bK_SL <<- v[,2],
        transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
        vector[2]: sigmaSSL ~ exponential(2),
        cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
        matrix[2,25]: zSSL ~ normal(0 , 1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_dS * dS ,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        aS_bar ~ normal(0,0.3),
        sigma_aS ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.1),
        sigma_S ~ exponential(1),
```

```
    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

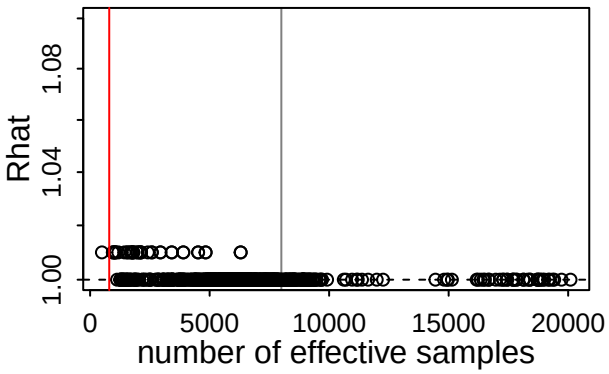
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

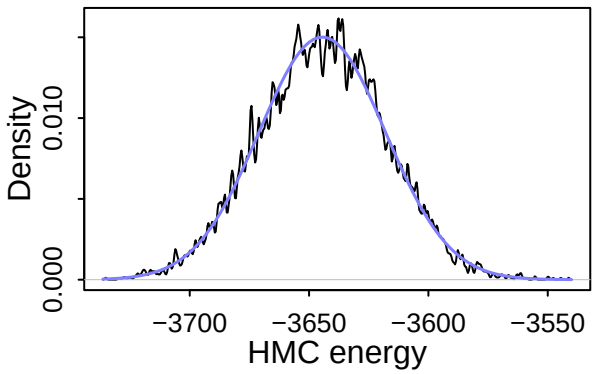
```
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
```

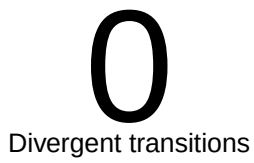
```
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
```

```
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 80.6 seconds.
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 81.3 seconds.
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 85.9 seconds.
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 174.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 105.5 seconds.
## Total execution time: 174.4 seconds.
```

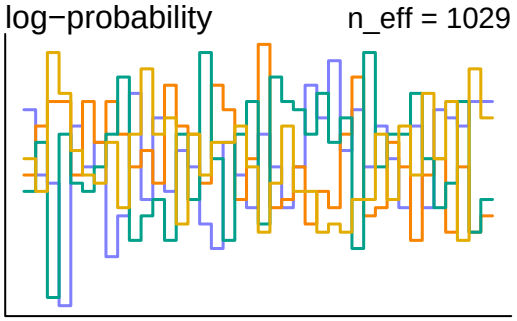
```
dashboard(m2.3)
```







Outlook good




```
precis(m2.3, depth=1)
```

```
##                mean          sd        5.5%       94.5%     rhat  ess_bulk
## aK_bar   -1.44780124 0.082891678 -1.58639862 -1.32192647 1.001313  2735.057
## sigma_aK  0.17432947 0.177706755  0.00953937  0.51042573 1.000095  8944.446
## bK_dK    -0.22200164 0.012973826 -0.24303496 -0.20105803 0.999946 14810.895
## U         0.02995852 0.002135841  0.02651220  0.03330642 1.004529  1129.324
## aS_bar   -0.75881535 0.034639696 -0.81239189 -0.70362758 1.000402  4892.492
## sigma_aS  0.99885987 0.239030940  0.64671907  1.40677972 1.000668  4372.770
## bS_dS    -0.19602995 0.017865577 -0.22475369 -0.16782141 1.001501 15135.862
## sigma_S   0.05229579 0.002814364  0.04800007  0.05697733 1.001662 11991.305
```

- This model is good enough.

Posterior Check

```
post <- extract.samples(m2.3)
```

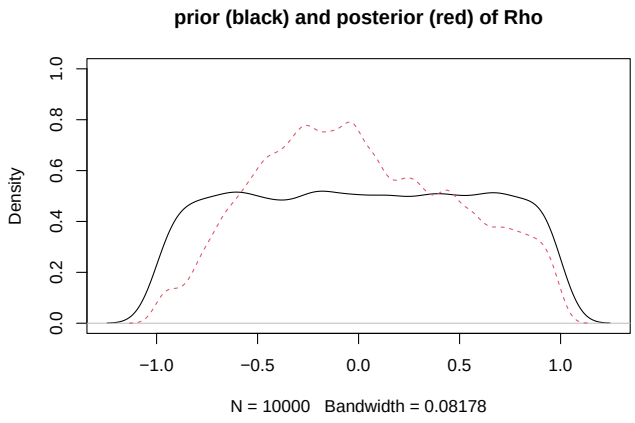
- I do not check the Hidden parameters since I have never touched it since m1

Correlation?

- Rho value shows a bit of correlation between S and SL.

```
R <- rlkjcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1), main = "prior (black) and posterior (red) of Rho")
# This is complicated as cholesky correlation
true_rho_samples=NULL
for (i in 1:2000) {
  L <- post$RhoSSL[i, , ]
  R <- L %*% t(L)
  true_rho_samples[i] <- R[1, 2]
```

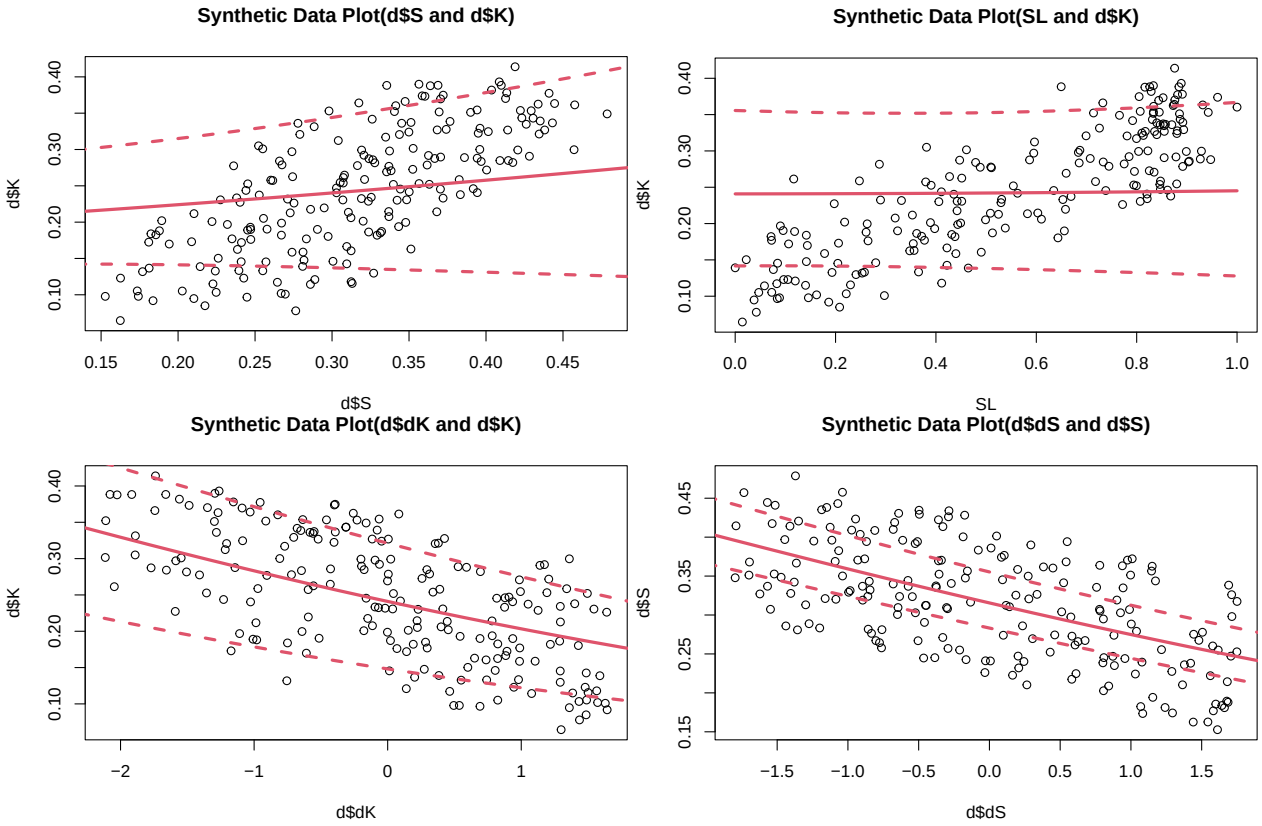
```
}
dens(true_rho_samples, add = TRUE, lty = 2, col = 2)
```



Posterior Predictions

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
    function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$dl
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
```

```
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



Overall K estimation is fine, but fit less than m1.3.

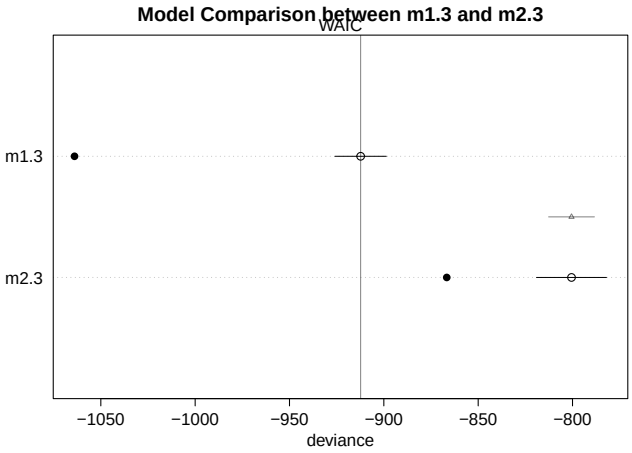
Let me compare those two models

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m2.3)
print(model_comparison)
```

```
##           WAIC       SE   dWAIC     dSE    pWAIC       weight
## m1.3 -912.2952 13.61934   0.000      NA 75.82207 1.000000e+00
## m2.3 -800.5361 18.55596 111.759 12.1456 33.05996 5.393084e-25
```

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m2.3")
```



Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

=> Therefore, the m1 is better than the other one.

Model 3

- Same thing! See Overview pdf for details.

```
m3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],

        # Basically same as m1 but not regularization (no pooling).
        # bK_T is regularized to maintain the efficiency.

        c(aK, bK_S,  bK_SL, bK_Y, bK_dK) ~ normal(0,0.1), # This is different from the other one, so th
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1), # regularization but not high sigma
        sigma_bK_T ~ exponential(1), # Needs to be 1 to maintain bK_T.
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

        # I am hoping/believing b_dS is negative, b_Y_S is positive, and bS_T is different for each one
        c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
        # bS_T is regularized.
```

```
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL   = "lower=0,upper=1", # SL is between 0 and 1
        Y    = "lower=0,upper=1" # Y is between 0 and 1
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1308:3: note:   by 'void ulam_cm
##  1308 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1308:3: note:   by 'void ulam_cm
##  1308 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crt
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~
## C:/Use
```

```
## rs/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1345:3: note:   by 'T_ ulam_cmdstanr_5
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1345:3: note:   by 'T_ ulam_cmds
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1345:3: note:   by 'T_ ulam_cmds
```

```
## const'
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp4UmqRk/model-b23c58d65b95.hpp:1345:3: note:   by 'T_ ulam_cmds
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 1

## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 2

## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 3

## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova

## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m

## Chain 4

## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
```

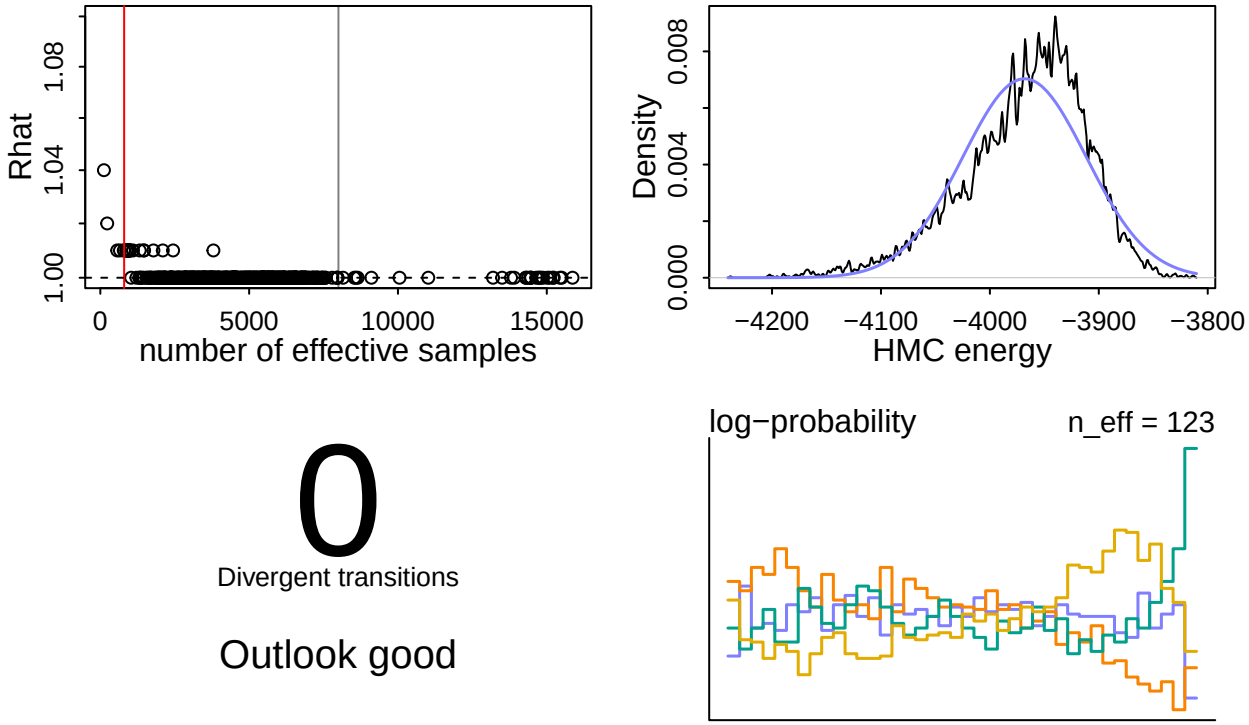
```
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
```

```
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
```

```
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 182.4 seconds.
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 190.1 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 192.2 seconds.
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 196.1 seconds.
```

```
##
## All 4 chains finished successfully.
## Mean chain execution time: 190.2 seconds.
## Total execution time: 196.2 seconds.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m3)
```



- without divergent but it has a few problem with efficiency. I will try to eliminate by removing parameters.

```
m3.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

        c(aK, bK_S,  bK_SL, bK_Y, bK_dK) ~ normal(0,0.1),
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1),
        sigma_bK_T ~ exponential(1),
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

        c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
```

```
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL   = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

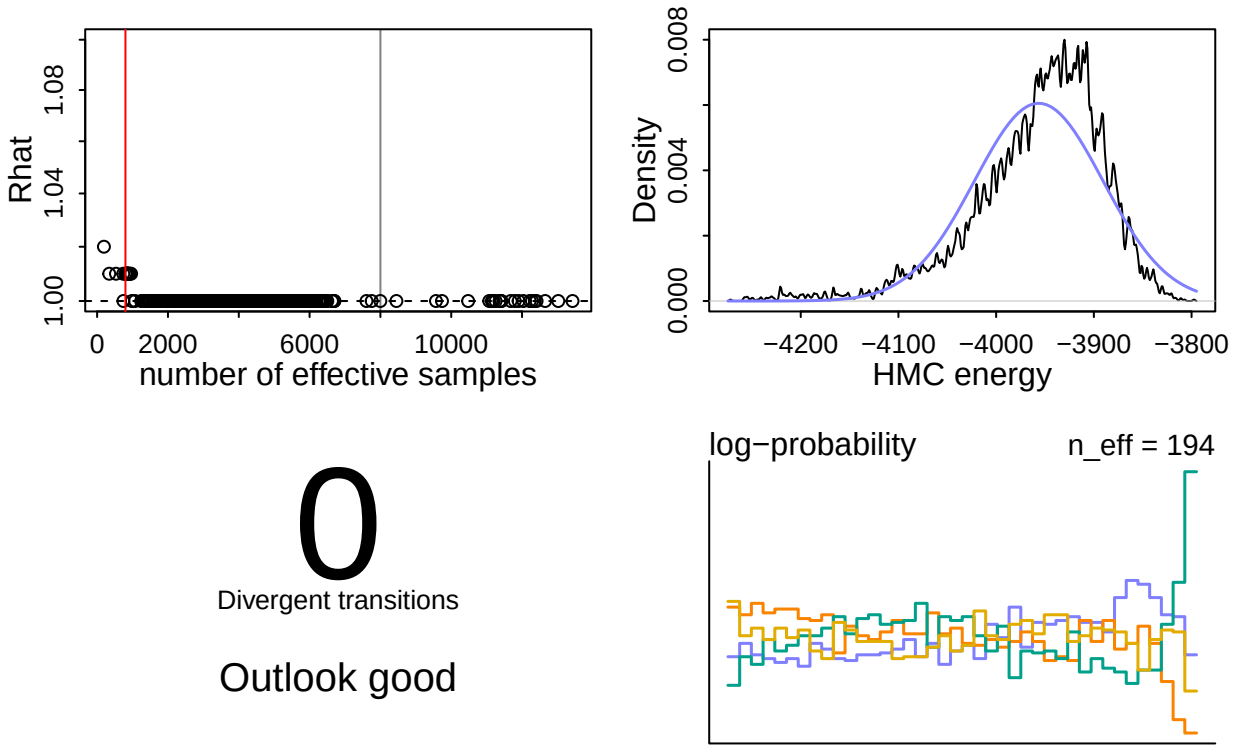
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
```

```
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
```

```
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
```

```
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 189.6 seconds.
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 218.8 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 220.7 seconds.
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 272.6 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 225.4 seconds.
## Total execution time: 272.7 seconds.
```

```
dashboard(m3.2)
```



- This is still not the best model with low efficiency. I will remove one more parameter.
- I can remove bK_T or bS_T but this time, I want to make this similar to m1.3 and m2.3.

```
m3.3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

        c(aK, bK_S,  bK_SL, bK_dK) ~ normal(0,0.1),
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1),
        sigma_bK_T ~ exponential(1),
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_T[T],

        c(aS, bS_dS) ~ normal(0,0.1),
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),

    # Model SL by Y
```

```
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL   = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

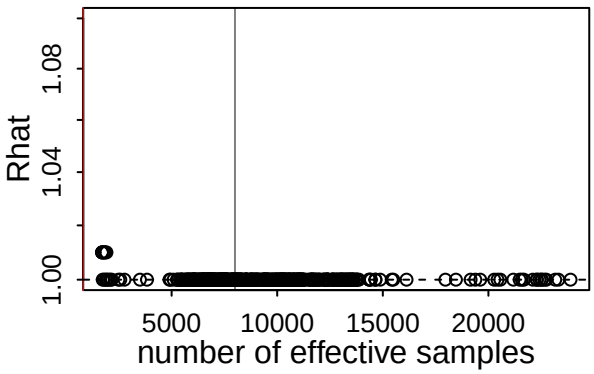
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
```

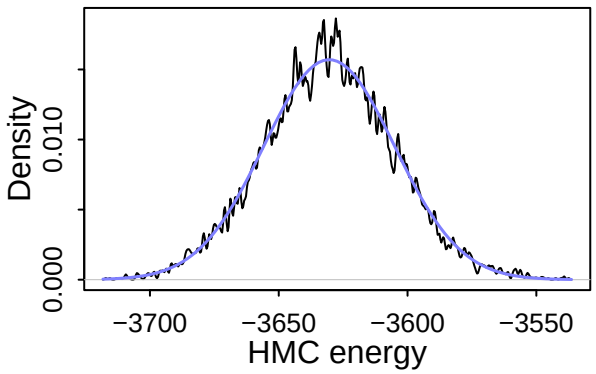
```
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
```

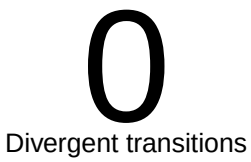
```
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
```

```
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 89.3 seconds.
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 92.4 seconds.
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 118.9 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 121.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 105.4 seconds.
## Total execution time: 121.1 seconds.
```

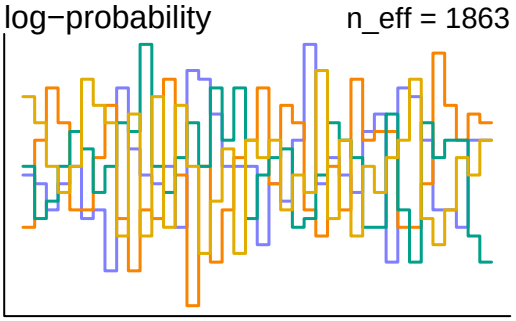
```
dashboard(m3.3)
```







Outlook good




```
precis(m3.3, depth = 1)
```

```
##                  mean          sd        5.5%       94.5%      rhat  ess_bulk
## bK_dK      -0.23865579 0.014950196 -0.26270104 -0.21501148 0.9999803 11513.044
## bK_SL       0.78359053 0.077620080  0.65534404  0.90591563 1.0010552  3501.422
## bK_S        0.31601726 0.097533458  0.16112005  0.47204576 1.0002963 15423.598
## aK         -0.17568227 0.103757362 -0.34007213 -0.00703161 0.9998737 13070.377
## bK_T_bar   -0.17401175 0.103906140 -0.34234824 -0.01038714 1.0006718 11666.782
## sigma_bK_T 10.73919319 1.782624067  8.10480606 13.76219758 1.0043803  2570.369
## U           0.03303019 0.003171059  0.02811273  0.03822726 1.0014399  2768.580
## bS_dS      -0.19457457 0.018081332 -0.22319882 -0.16575858 1.0019337 19156.539
## aS         -0.33802476 0.079106666 -0.46227887 -0.20917064 1.0003912 10610.044
## bS_T_bar   -0.33751966 0.078139886 -0.46028042 -0.21214349 1.0003943  9931.875
## sigma_bS_T  1.73345173 0.572188302  1.03642729  2.72079047 1.0007594  2480.719
## sigma_S     0.05169215 0.002648891  0.04761566  0.05611398 1.0007672 13286.015
```

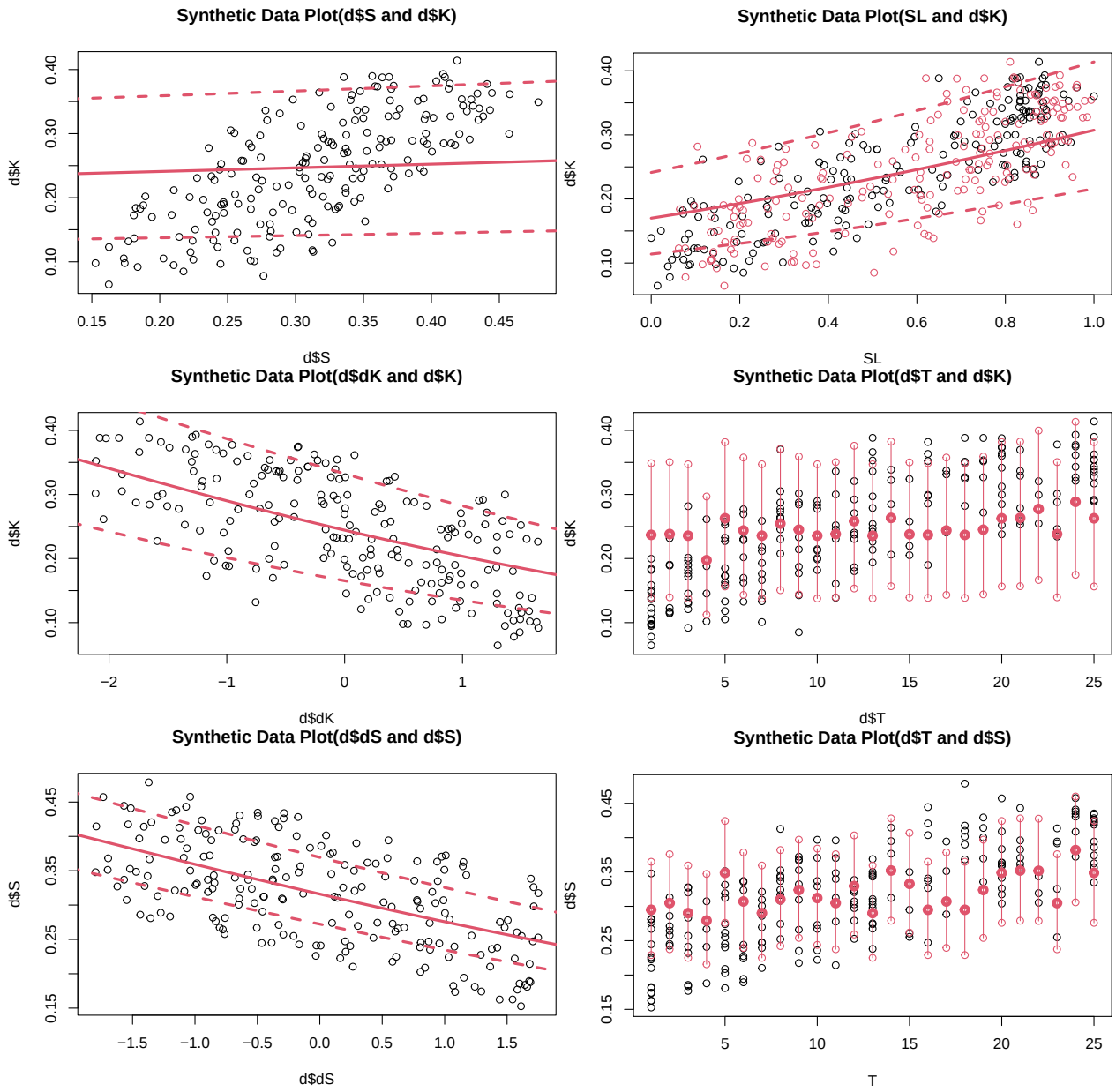
Posterior Check

```
post <- extract.samples(m3.3)
```

- Again, I do not check the Hidden parameters since I have never touched it since m1

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
    function (i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```

```
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
points(post$SL[1,], d$K, col = 2) # include the estimated points, though we confirmed above that it is
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_dK * d$dK + colMeans(pos
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * i
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
KbyT<- sapply(1:25,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + post$bS_dS * i + colMeans(post$bS_T)[d$T] )) # Used mean of each s
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$S, main = "Synthetic Data Plot(d$T and d$S)")
SbyT<- sapply(1:25,
    function (i) inv_logit(post$aS + post$bS_dS * d$dS + colMeans(post$bS_T)[i]))
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)
```



I have no idea why it does not model bK_T and bS_T very well... I will figure it out. Other than that, looks great, let me compare all those models.

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m3.3)
print(model_comparison)
```

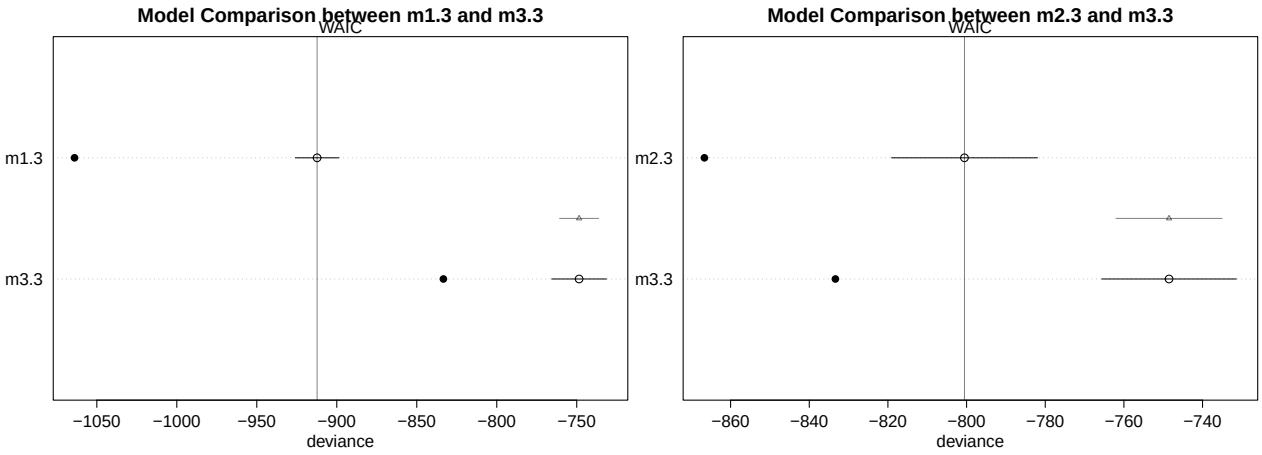
```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m1.3 -912.2952 13.61934   0.0000       NA 75.82207 1.000000e+00
## m3.3 -748.5199 17.13189 163.7753 12.22739 42.41673 2.733078e-36
```

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m3.3")
```

```r
model_comparison <- compare(m2.3, m3.3)
print(model_comparison)
```

```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m2.3 -800.5361 18.55596  0.00000       NA 33.05996 1.000000e+00
## m3.3 -748.5199 17.13189 52.01625 13.49928 42.41673 5.067747e-12
```

```r
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m2.3 and m3.3")
```



This looks Model 1 is a bit better in this case.

Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K) may be examined with actual data.
- The Model 3 still has a problem of bK_T and bS_T as they do not fit the data. I am trying to understand the reason and fix it.

```r
rm(list=ls())
```